

OPERATING SYSTEM

Important Notes for Examination & Assignment Preparation

Topics Covered

1. Introduction to Operating System
2. Process Management
3. CPU Scheduling
4. Process Synchronization
5. Memory Management
6. File System

Quick Exam Tip: Learn definitions, objectives, diagrams, algorithms, advantages/disadvantages, and comparison tables. For numerical problems, practice FCFS, SJF, Priority, Round Robin, and memory-allocation examples.

1. Introduction to Operating System

Definition

An Operating System (OS) is system software that acts as an intermediary between the user/application programs and computer hardware. It manages hardware resources and provides services for program execution.

Main Objectives

- Convenience: makes the computer easier to use.
- Efficiency: uses CPU, memory, storage, and I/O devices effectively.
- Resource management: allocates resources among processes and users.
- Protection and security: controls access to system resources.
- Ability to evolve: supports new hardware, applications, and features.

Major Functions of an OS

- Process management
- CPU scheduling
- Memory management
- File-system management
- I/O and device management
- Protection and security
- Networking
- Error detection and accounting

Types of Operating Systems

- Batch OS – jobs are collected and processed in batches.
- Multiprogramming OS – several programs are kept in memory so CPU utilization increases.
- Multitasking/Time-sharing OS – CPU switches rapidly among tasks to provide interactive use.
- Multiprocessing OS – uses two or more processors/cores.
- Real-time OS – provides responses within specified timing constraints.
- Distributed OS – coordinates resources across multiple networked computers.

Kernel and System Calls

The kernel is the core part of an OS that runs with high privilege and manages hardware and system resources. System calls provide a controlled interface through which user programs request OS services, such as process creation, file access, memory allocation, and device operations.

2. Process Management

Process

A process is a program in execution. A program is passive code stored on storage, whereas a process is an active entity with a current state, program counter, registers, stack, data, and allocated resources.

Process States

- New – process is being created.

- Ready – process is waiting to be assigned the CPU.
- Running – instructions are currently being executed.
- Waiting/Blocked – process is waiting for an event or I/O completion.
- Terminated – process has finished execution.

Process Control Block (PCB)

PCB is the OS data structure containing information needed to manage a process. Typical fields include process ID, process state, program counter, CPU registers, scheduling information, memory-management information, accounting information, and I/O status.

Context Switch

A context switch occurs when the CPU changes from one process/thread to another. The OS saves the current process state in its PCB and loads the saved state of the next process. Context switching is necessary but adds overhead because it does not directly perform useful application work.

Threads

A thread is a lightweight unit of CPU execution within a process. Threads of the same process normally share code, data, and files while having their own program counter, registers, and stack. Benefits include responsiveness, resource sharing, economy, and improved parallelism on multicore systems.

Inter-Process Communication (IPC)

- Shared memory – processes communicate through a shared memory region; synchronization is usually required.
- Message passing – processes exchange messages using send/receive operations.
- Pipes, sockets, signals, and other OS mechanisms can also support communication.

3. CPU Scheduling

Definition

CPU scheduling selects a process from the ready queue and assigns the CPU to it. The scheduler aims to use the CPU efficiently while providing good response and turnaround performance.

Scheduling Criteria

- CPU utilization – percentage of time CPU is busy; generally maximize.
- Throughput – number of processes completed per unit time; generally maximize.
- Turnaround time – completion time – arrival time; generally minimize.
- Waiting time – total time spent waiting in the ready queue; generally minimize.
- Response time – first CPU response time – arrival time; generally minimize.

Preemptive vs Non-preemptive

In preemptive scheduling, the OS can take the CPU away from a running process. In non-preemptive scheduling, a process keeps the CPU until it terminates or voluntarily blocks. Preemption can improve responsiveness but introduces context-switch overhead and synchronization concerns.

FCFS (First-Come, First-Served)

Processes are executed in order of arrival. It is simple and non-preemptive. A disadvantage is the convoy effect, where short processes may wait behind a long process.

SJF (Shortest Job First)

The process with the smallest next CPU burst is selected. It can minimize average waiting time when burst lengths are known or estimated. SJF is normally described as non-preemptive.

SRTF (Shortest Remaining Time First)

SRTF is the preemptive form of SJF. If a newly arrived process has a shorter remaining CPU burst than the currently running process, the CPU may be preempted.

Priority Scheduling

The process with the highest priority is selected. It may be preemptive or non-preemptive. Starvation can occur for low-priority processes; aging gradually increases the priority of waiting processes to reduce starvation.

Round Robin (RR)

Each ready process receives a fixed time quantum in cyclic order. If it does not finish within the quantum, it is preempted and moved to the end of the ready queue. Small quanta improve responsiveness but can increase context-switch overhead.

Important Formulae

- Turnaround Time (TAT) = Completion Time – Arrival Time
- Waiting Time (WT) = Turnaround Time – Burst Time
- Response Time = First CPU Start Time – Arrival Time
- Average WT = Sum of all waiting times / Number of processes
- Average TAT = Sum of all turnaround times / Number of processes

Gantt Chart

A Gantt chart shows the order and time intervals in which processes use the CPU. For numerical problems, first construct the Gantt chart, determine completion times, then calculate TAT and WT using the formulae above.

4. Process Synchronization

Why Synchronization is Needed

When multiple processes or threads access shared data concurrently, their operations can interleave and produce incorrect results. Synchronization ensures correct ordering and mutual exclusion where required.

Race Condition

A race condition occurs when the outcome depends on the timing or order of concurrent accesses to shared data. The critical-section problem addresses safe access to shared resources.

Critical Section

A critical section is the part of a process that accesses shared data or resources. A correct solution should satisfy mutual exclusion, progress, and bounded waiting.

Mutex

A mutex (mutual exclusion lock) allows only one thread/process at a time to enter a protected critical section. A thread locks before entering and unlocks after leaving.

Semaphore

A semaphore is an integer synchronization primitive accessed through atomic wait (P) and signal (V) operations. A binary semaphore has values typically used for mutual exclusion; a counting semaphore can represent multiple identical resources.

Monitor

A monitor is a high-level synchronization construct that encapsulates shared data and procedures. Mutual exclusion is automatically provided for monitor procedures, while condition variables can support waiting and signaling.

Classical Problems

- Producer–Consumer (Bounded Buffer): coordinates producers adding items and consumers removing items from a finite buffer.
- Readers–Writers: manages concurrent readers and writers accessing shared data.
- Dining Philosophers: illustrates deadlock and resource-allocation issues.

Deadlock

Deadlock is a state in which a set of processes are permanently waiting for events/resources that can only be caused/released by other processes in the same set.

Four Necessary Conditions for Deadlock

- Mutual exclusion
- Hold and wait
- No preemption
- Circular wait

Deadlock Handling

- Prevention – design the system so at least one necessary condition cannot hold.
- Avoidance – allocate resources only when the resulting state remains safe; Banker's algorithm is a classic method.
- Detection and recovery – allow deadlock, detect it, then recover by termination or resource preemption.

5. Memory Management

Definition

Memory management controls the allocation and deallocation of main memory and keeps track of which memory areas are in use. It also supports address translation and protection.

Logical and Physical Address

A logical (virtual) address is generated by the CPU. A physical address refers to the actual location in main memory. The Memory Management Unit (MMU) translates addresses during execution.

Contiguous Memory Allocation

A process is allocated a contiguous region of physical memory. Common strategies include first-fit, best-fit, and worst-fit. External fragmentation can occur when free memory is split into small holes.

Paging

Paging divides logical memory into fixed-size pages and physical memory into fixed-size frames. A page table maps pages to frames. Paging removes external fragmentation but can cause internal fragmentation.

Segmentation

Segmentation divides a program into logical variable-sized units such as code, data, and stack. A logical address consists of a segment number and offset. Segmentation supports logical protection and sharing but can suffer from external fragmentation.

Virtual Memory

Virtual memory allows processes to execute even when their entire address space is not in physical memory. It uses secondary storage as a backing store and loads required pages/segments when needed.

Demand Paging

With demand paging, a page is loaded into memory only when it is referenced. If the required page is absent, a page fault occurs and the OS brings the page from secondary storage into a free frame.

Page Replacement Algorithms

- FIFO – replaces the page that entered memory first; simple but may show Belady's anomaly.
- Optimal – replaces the page that will not be used for the longest future time; gives the theoretical minimum page faults but future references are not known in practice.
- LRU – replaces the page that has not been used for the longest time; often performs well but requires tracking usage.

Thrashing

Thrashing occurs when excessive page faults cause the system to spend most of its time swapping pages rather than executing useful work. It can result from insufficient frames and excessive multiprogramming.

Fragmentation

Internal fragmentation is wasted space inside an allocated block. External fragmentation is free space scattered between allocated blocks. Paging largely avoids external fragmentation but may have internal fragmentation.

6. File System

Definition

A file system organizes, stores, names, protects, and retrieves files on secondary storage. It provides an abstraction that lets users and programs access data without managing raw disk blocks directly.

File

A file is a named collection of related information stored on secondary storage. File attributes can include name, identifier, type, location, size, protection, timestamps, and ownership information.

File Operations

- Create
- Open
- Read
- Write
- Seek/reposition
- Close
- Delete
- Truncate

File Access Methods

- Sequential access – records/bytes are accessed in order.
- Direct (random) access – any block or record can be accessed directly using its location.

Directory Structures

- Single-level directory – all files are in one directory.

- Two-level directory – separate directory for each user.
- Tree-structured directory – hierarchical directories and subdirectories.
- Acyclic-graph directory – allows shared files/directories without cycles.

File Allocation Methods

- Contiguous allocation – file blocks are stored consecutively; fast access but may cause external fragmentation and file-growth problems.
- Linked allocation – each file block points to the next; avoids external fragmentation but is poor for direct access.
- Indexed allocation – an index block contains pointers to file blocks; supports direct access and flexible allocation.

Free-Space Management

The file system tracks unused disk blocks using methods such as bitmaps/bit vectors, linked lists, grouping, and counting.

File Protection

Protection mechanisms control who can read, write, execute, or modify files. Common concepts include access-control lists, permissions, ownership, and authentication.

Disk Scheduling – Important Algorithms

- FCFS – services disk requests in arrival order.
- SSTF – selects the request with the shortest seek distance from the current head position.
- SCAN – disk arm moves in one direction servicing requests, then reverses direction.
- C-SCAN – services requests in one direction and returns to the beginning without servicing on the return path.
- LOOK/C-LOOK – like SCAN/C-SCAN but reverses/jumps at the last pending request rather than the physical disk end.

Quick Revision Sheet

Topic	Must Remember
OS	Resource manager + service provider + interface between user/programs and hardware
Process	Program in execution; states: New, Ready, Running, Waiting, Terminated
PCB	Stores process state, PC, registers, scheduling, memory and I/O information
CPU Scheduling	FCFS, SJF/SRTF, Priority, Round Robin; know Gantt charts and formulae
Synchronization	Race condition, critical section, mutex, semaphore, monitor
Deadlock	Mutual exclusion, hold & wait, no preemption, circular wait
Memory	Paging, segmentation, virtual memory, demand paging, page replacement
File System	File operations, directories, allocation methods, free-space management

High-priority numerical practice: CPU scheduling (FCFS, SJF, SRTF, Priority, Round Robin), page replacement (FIFO, Optimal, LRU), and memory allocation (first-fit, best-fit, worst-fit).

High-priority theory: process states and PCB, system calls, threads, critical-section requirements, semaphores, deadlock conditions, paging/virtual memory, file allocation and directory structures.